ЗВІТ

з лабораторної роботи №2 з дисципліни «Системне програмування»

Тема: Виконання дій у системі за допомогою скриптів (Batch / CMD)

1. Мета роботи

- навчитись автоматизувати системні операції за допомогою пакетних файлів (batch-файлів);
- ознайомитися з внутрішніми та зовнішніми командами Windows CMD;
- навчитися працювати з аргументами командного рядка та лог-файлами;
- отримати практичні навички аналізу процесів, роботи з файлами та каталогами, архівування, перевірки мережної доступності, взаємодії з іншими ПК, моніторингу стану системи.

2. Опис програми та її призначення

Розроблено пакетний файл lab.cmd, який автоматизує рутинні операції системного адміністрування Windows: ведення журналу подій (log-файл), синхронізацію системного часу з NTP-сервером, аналіз та завершення процесів, очищення тимчасових файлів, регулярне архівування каталогу з подальшим копіюванням архіву, контроль застарілих архівів, перевірку мережної доступності (Internet, віддалений ПК, список ПК локальної мережі, перелік IP з файлу), контроль розміру log-файлу та вільного місця на дисках, збереження даних systeminfo, а також підготовку до автозапуску через Планувальник завдань Windows.

Скрипт приймає 6 аргументів командного рядка:

Аргумент	**Позначення у скрипті**	**Значення**
%1	szLogFile	назва log-файлу
%2	szSourceDir	каталог з файлами для регулярного архівування
%3	szProcessName	назва процесу, який потрібно завершити
%4	szArchiveDir	каталог, куди копіюється створений архів
%5	szRemoteIP	IP-адреса комп'ютера, який потрібно вимкнути
%6	nMaxLogKB	максимальний розмір log-файлу, КВ

Приклад запуску:

```
lab.cmd lab.log d:\baza xx.exe d:\archive 192.168.1.2 500
```

Примітка щодо мови повідомлень: усі рядки, що виводяться скриптом (echo, коментарі REM), навмисно написано англійською мовою (лише символи ASCII). Це зроблено для повної сумісності з кодовою сторінкою консолі Windows: кирилиця у пакетних файлах вимагає узгодження кодування файлу та кодової сторінки cmd.exe (chcp), а на файлах такого розміру ще й може провокувати відомий збій cmd.exe при переключенні на UTF-8 (chcp 65001) через розбиття багатобайтових символів на межі 4096 байт внутрішнього буфера. Використання лише ASCII-символів повністю усуває цю категорію проблем незалежно від мовних і регіональних налаштувань комп'ютера, на якому запускається скрипт.

3. Глобальні змінні скрипта

Оскільки скрипт написаний мовою batch, а не C, до імен змінних застосована угорська нотація в адаптованому вигляді (без типів, характерних для C, оскільки batch не має статичної типізації):

Змінна	**Призначення**
szLogFile, szSourceDir, szProcessName, szArchiveDir, szRemoteIP	рядкові параметри, отримані з аргументів командного рядка (префікс sz — рядок, що закінчується нулем)
nMaxLogKB, nDeleted, nLogSizeKB, nOldFound	числові значення (префікс n — ціле)
dtNow, dtOld, dtNew	мітки часу у вигляді рядка (префікс dt — дата/час)
boolFound, boolAllUp	логічні прапорці 0/1
szSmtpServer, szSmtpUser, szSmtpPass, szSmtpFrom, szSmtpTo, nSmtpPort	налаштування відправки пошти через Send-MailMessage
szScriptDir, szIpListFile	службові шляхи (каталог скрипта, файл зі списком IP)

4. Методи та прийоми, використані при вирішенні задачі

- Модульна структура: кожен з 16 кроків завдання оформлено як окрему підпрограму (call :StepXX_Назва), що відповідає вимозі описувати структуру програми частинами.
- Спільні допоміжні підпрограми :GetTimeStamp та :SendMail винесені окремо, щоб уникнути дублювання коду.
- Використання setlocal EnableDelayedExpansion та !змінна! для коректного читання значень змінних, які змінюються всередині циклів for та блоків if.
- Виклик PowerShell (powershell -NoProfile -Command ...) для операцій, які незручно або неможливо виконати чистими засобами CMD: формат дати/часу, обчислення розміру файлу, архівування (Compress-Archive), відображення дискового простору (Get-PSDrive), відправка пошти (Send-MailMessage).

- Перенаправлення виводу (>> "%szLogFile%" 2>&1) для журналювання результату кожного кроку, включно з повідомленнями про помилки.
- Перевірка кодів завершення (errorlevel) після кожної команди, від якої залежить подальша логіка (ping, taskkill, w32tm, shutdown).
- Захисні перевірки exist перед операціями з файлами та каталогами, щоб уникнути аварійного завершення скрипта.

5. Опис структури програми (по кроках)

Крок 1. Перевірка/створення log-файлу, запис часу відкриття.

Крок 2. Синхронізація часу командою w32tm /resync, запис старого/нового часу та статусу.

Крок 3. Запис у log повного списку процесів (tasklist).

Крок 4. Пошук процесу за іменем (Аргумент3) через tasklist /FI, завершення через taskkill /IM ... /F, журналювання результату.

Крок 5. Видалення файлів *.tmp та temp*.* у каталозі Аргумент2, підрахунок видалених файлів через цикл for.

Крок 6. Формування мітки часу через PowerShell, стиснення каталогу Аргумент2 у ZIP командою Compress-Archive, копіювання архіву у каталог Аргумент4.

Крок 7. Обчислення вчорашньої дати через PowerShell, перевірка наявності архіву за цю дату в Аргумент4; за відсутності — запис у log та виклик :SendMail.

Крок 8. Підрахунок і видалення архівів, старших за 30 днів, командою forfiles /D -30 /M *.zip /C "cmd /c del @file".

Крок 9. Перевірка інтернет-з'єднання через ping 8.8.8.8.

Крок 10. Перевірка доступності ПК за IP (Аргумент5) через ping; за доступності — віддалене вимкнення командою shutdown /m \\IP /s /t 0.

Крок 11. Отримання списку ПК мережі командами net view та arp -a.

Крок 12. Почергова перевірка (ping -n 1) кожної IP-адреси з файлу ipon.txt; за наявності недоступних — запис у log та відправка листа.

Крок 13. Обчислення поточного розміру log-файлу (PowerShell, Get-Item.Length), порівняння з лімітом Аргумент6, за перевищення — лист.

Крок 14. Виведення вільного/зайнятого місця на всіх дисках через PowerShell Get-PSDrive -PSProvider FileSystem.

Крок 15. Збереження результату systeminfo у файл systeminfo-YYYYMMDD-HHMMSS.txt в каталозі скрипта.

Крок 16. Виведення у log приклад команди schtasks /create для автозапуску скрипта через Планувальник завдань.

6. Теоретичні відомості

6.1. Пакетні файли Windows (Batch / CMD)

- Синтаксис: команди виконуються послідовно, рядок, що починається з REM або ::, є коментарем; @echo off вимикає відображення самих команд.

- Параметри командного рядка позначаються %1...%9; %~1 прибирає лапки навколо значення, %~dp0 повертає диск та шлях до каталогу самого скрипта.
- Умовні оператори: if, if exist / if not exist, if defined; для змінних, що обчислюються в циклі, потрібне відкладене розкриття (setlocal EnableDelayedExpansion, символи !var!).
- Цикли: for %%F in (набір) do ... — перебір файлів/значень; for /f — перебір рядків виводу команди або текстового файлу; for /l — числовий цикл.
- Перенаправлення: > записує (перезаписуючи) вивід у файл, >> дописує у кінець файлу, 2>&1 перенаправляє потік помилок туди ж, куди й стандартний вивід.
- Оператори порівняння в IF: EQU, NEQ, LSS, LEQ, GTR, GEQ (для чисел) та звичайне порівняння рядків через ==.

6.2. Використані зовнішні (системні) команди

Команда	**Призначення**
tasklist	виводить список запущених процесів; з ключем /FI дозволяє фільтрувати за іменем образу
taskkill /IM <ім'я> /F	примусово завершує процес за іменем виконуваного файлу
ping	перевіряє доступність вузла мережі за IP або іменем, використовується як для Internet, так і для локальних хостів
net view	виводить список комп'ютерів, доступних у поточному домені/робочій групі
arp -a	показує таблицю відповідності IP- та MAC-адрес пристроїв, з якими нещодавно була взаємодія
shutdown /m \\IP /s /t 0	надсилає команду вимкнення віддаленому комп'ютеру (потребує прав адміністратора та дозволеного віддаленого доступу)
wmic logicaldisk get size,freespace,caption	класичний спосіб отримати розмір і вільне місце дисків (у сучасних Windows частково замінюється PowerShell)
systeminfo	виводить детальну інформацію про конфігурацію ОС та апаратне забезпечення
forfiles /D -30 /P <каталог> /M *.zip /C "cmd /c del @file"	знаходить та обробляє (тут — видаляє) файли за критерієм дати модифікації, старші вказаної кількості днів
w32tm /resync	примушує службу часу Windows негайно синхронізуватися з налаштованим NTP-сервером

6.3. PowerShell у batch-файлах

Команди PowerShell викликаються з CMD через powershell -NoProfile -Command "...". Це дозволяє використовувати можливості, яких немає у чистому CMD: Compress-Archive (створення ZIP-архівів), Get-PSDrive -PSProvider FileSystem (інформація про диски), Get-Date -Format (гнучке форматування дати/часу незалежно від регіональних налаштувань), Get-Item.Length (точний розмір файлу в байтах) та Send-MailMessage (відправка листів через SMTP).

6.4. NTP та синхронізація часу

Windows Time Service (w32time) відповідає за синхронізацію системного годинника з NTP-сервером. Команда w32tm /resync ініціює негайну синхронізацію; альтернативою є net time /set, що працює через застарілий протокол NetBIOS Time і в сучасних мережах використовується рідко.

6.5. Архівація

Для архівування каталогу використано вбудований у PowerShell командлет Compress-Archive, який не потребує встановлення стороннього ПЗ. Як альтернативу документ до лабораторної допускає використання зовнішніх утиліт 7-Zip або WinRAR через їх командний рядок (7z.exe a архів.zip каталог*), якщо вони встановлені в системі.

6.6. Перевірка дискового простору

Класичний варіант — wmic logicaldisk get size,freespace,caption, що виводить розмір і вільне місце для кожного логічного диска в байтах. У скрипті натомість застосовано PowerShell-еквівалент Get-PSDrive -PSProvider FileSystem, оскільки утиліта wmic позначена застарілою (deprecated) починаючи з останніх версій Windows 10/11.

7. Текст batch-файлу (lab.cmd) з коментарями

```
@echo off
REM ============================================================
REM File       : lab.cmd
REM Purpose    : Automation of Windows system administration tasks
REM              (logging, time sync, process analysis, cleanup,
REM              archiving, network diagnostics, remote shutdown,
REM              disk space check, systeminfo, task scheduler).
REM Author     : Student Zinchenko Maksym (Lab Work #2, System Programming)
REM ============================================================
setlocal EnableExtensions EnableDelayedExpansion


REM ------------------------------------------------------------
REM If szSmtpServer is left empty, sending is
REM skipped and a warning is written to the log instead.
REM ------------------------------------------------------------
set "szSmtpServer="
set "nSmtpPort=587"
set "szSmtpUser="
set "szSmtpPass="
set "szSmtpFrom="
set "szSmtpTo="


REM ------------------------------------------------------------
REM Argument count check
REM ------------------------------------------------------------
if "%~6"=="" (
  echo Error: 6 arguments are expected.
```

```
  echo Usage: lab.cmd ^<log^> ^<dirToArchive^> ^<processName^> ^<archiveDir^> ^<remoteIP^> ^<maxLogKB^>
  exit /b 1
)

REM ------------------------------------------------------------
REM Named argument variables (Hungarian notation)
REM ------------------------------------------------------------
set "szLogFile=%~1"
set "szSourceDir=%~2"
set "szProcessName=%~3"
set "szArchiveDir=%~4"
set "szRemoteIP=%~5"
set "nMaxLogKB=%~6"
set "szScriptDir=%~dp0"
set "szIpListFile=%szScriptDir%ipon.txt"

call :Step01_InitLog
call :Step02_SyncTime
call :Step03_ListProcesses
call :Step04_KillProcess
call :Step05_CleanTempFiles
call :Step06_CompressAndCopy
call :Step07_CheckYesterdayArchive
call :Step08_PurgeOldArchives
call :Step09_CheckInternet
call :Step10_CheckAndShutdownRemote
call :Step11_ListNetworkHosts
call :Step12_CheckIpList
call :Step13_CheckLogSize
call :Step14_CheckDiskSpace
call :Step15_SaveSysteminfo
call :Step16_ShowSchedulerHint

echo Script finished. See "%szLogFile%".
endlocal
exit /b 0



REM ============================================================
REM Step 1: Check / create the log file
REM ============================================================
:Step01_InitLog
  if not exist "%szLogFile%" (
    type nul > "%szLogFile%"
  )
  call :GetTimeStamp dtNow
  (
    echo [!dtNow!] Log file "%szLogFile%" opened or created.
  )>> "%szLogFile%"
  exit /b 0



REM ============================================================
REM Step 2: NTP time synchronization (auto-starts w32time if needed)
REM ============================================================
:Step02_SyncTime
  call :GetTimeStamp dtOld


  REM Make sure the Windows Time service is running; w32tm needs it.
  sc query w32time | find "RUNNING" >nul 2>&1
  if !errorlevel! NEQ 0 (
    net start w32time >> "%szLogFile%" 2>&1
  )
```

```
  w32tm /resync >> "%szLogFile%" 2>&1
  if !errorlevel! EQU 0 (
    set "szSyncStatus=success"
  ) else (
    set "szSyncStatus=FAILED (needs administrator rights / w32time service)"
  )
  call :GetTimeStamp dtNew
  (
    echo [!dtNew!] Time synchronization: !szSyncStatus!.
    echo [!dtNew!]   Old time: !dtOld!
    echo [!dtNew!]   New time: !dtNew!
  )>> "%szLogFile%"
  exit /b 0



REM ============================================================
REM Step 3: List running processes
REM ============================================================
:Step03_ListProcesses
  call :GetTimeStamp dtNow
  (
    echo [!dtNow!] List of running processes ^(tasklist^):
    tasklist
  )>> "%szLogFile%"
  exit /b 0



REM ============================================================
REM Step 4: Terminate the process named in Argument3
REM ============================================================
:Step04_KillProcess
  call :GetTimeStamp dtNow
  tasklist /FI "IMAGENAME eq %szProcessName%" 2>nul | find /I "%szProcessName%" >nul
  if !errorlevel! EQU 0 (
    taskkill /IM "%szProcessName%" /F >> "%szLogFile%" 2>&1
    if !errorlevel! EQU 0 (
      echo [!dtNow!] Process %szProcessName% was found, terminated successfully.>> "%szLogFile%"
    ) else (
      echo [!dtNow!] Process %szProcessName% was found, but termination FAILED.>> "%szLogFile%"
    )
  ) else (
    echo [!dtNow!] Process %szProcessName% was not found among running processes.>> "%szLogFile%"
  )
  exit /b 0



REM ============================================================
REM Step 5: Delete *.tmp and temp*.* files in Argument2 directory
REM ============================================================
:Step05_CleanTempFiles
  set /a nDeleted=0
  if exist "%szSourceDir%\" (
    for %%F in ("%szSourceDir%\*.tmp") do (
      del /Q "%%F" >nul 2>&1
      if !errorlevel! EQU 0 set /a nDeleted+=1
    )
    for %%F in ("%szSourceDir%\temp*.*") do (
      if exist "%%F" (
        del /Q "%%F" >nul 2>&1
        if !errorlevel! EQU 0 set /a nDeleted+=1
      )
    )
  )
  call :GetTimeStamp dtNow
  echo [!dtNow!] Temporary files deleted: !nDeleted! ^(from %szSourceDir%^).>> "%szLogFile%"
```

```
  exit /b 0



REM ============================================================
REM Step 6: Compress Argument2 directory into a ZIP, copy to Argument4
REM ============================================================
:Step06_CompressAndCopy
  call :GetTimeStamp dtNow
  for /f %%T in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd-HHmmss"') do set "szStamp=%%T"
  set "szZipName=%szStamp%.zip"

  if not exist "%szArchiveDir%\" mkdir "%szArchiveDir%" >nul 2>&1

  powershell -NoProfile -Command ^
    "Compress-Archive -Path '%szSourceDir%\*' -DestinationPath '%szScriptDir%%szZipName%' -Force" ^
    >> "%szLogFile%" 2>&1

  if exist "%szScriptDir%%szZipName%" (
    copy /Y "%szScriptDir%%szZipName%" "%szArchiveDir%\%szZipName%" >nul 2>&1
    echo [!dtNow!] Archive %szZipName% created and copied to %szArchiveDir%.>> "%szLogFile%"
  ) else (
    echo [!dtNow!] Failed to create archive %szZipName%.>> "%szLogFile%"
  )
  exit /b 0



REM ============================================================
REM Step 7: Check whether yesterday's archive exists
REM ============================================================
:Step07_CheckYesterdayArchive
  call :GetTimeStamp dtNow
  for /f %%D in ('powershell -NoProfile -Command "(Get-Date).AddDays(-1).ToString('yyyyMMdd')"') do set
"szYesterday=%%D"

  set "boolFound=0"
  if exist "%szArchiveDir%\" (
    for %%F in ("%szArchiveDir%\%szYesterday%-*.zip") do set "boolFound=1"
  )

  if "!boolFound!"=="1" (
    echo [!dtNow!] Archive for the previous day ^(!szYesterday!^) was found.>> "%szLogFile%"
  ) else (
    echo [!dtNow!] Archive for the previous day ^(!szYesterday!^) is MISSING.>> "%szLogFile%"
    call :SendMail "Lab2: missing archive" "No archive for !szYesterday! found in %szArchiveDir%."
  )
  exit /b 0



REM ============================================================
REM Step 8: Delete archives older than 30 days in Argument4
REM ============================================================
:Step08_PurgeOldArchives
  call :GetTimeStamp dtNow
  set /a nOldFound=0
  if exist "%szArchiveDir%\" (
    for /f %%C in ('forfiles /P "%szArchiveDir%" /M *.zip /D -30 2^>nul ^| find /c /v ""') do set
"nOldFound=%%C"
    forfiles /P "%szArchiveDir%" /D -30 /M *.zip /C "cmd /c del @file" >nul 2>&1
  )
  echo [!dtNow!] Old archives found and deleted ^(older than 30 days^): !nOldFound!.>> "%szLogFile%"
  exit /b 0
```

```
REM ============================================================
REM Step 9: Check Internet connectivity
REM ============================================================
:Step09_CheckInternet
  call :GetTimeStamp dtNow
  ping -n 2 8.8.8.8 >nul 2>&1
  if !errorlevel! EQU 0 (
    echo [!dtNow!] Internet connection: available ^(ping 8.8.8.8 succeeded^).>> "%szLogFile%"
  ) else (
    echo [!dtNow!] Internet connection: NOT available.>> "%szLogFile%"
  )
  exit /b 0



REM ============================================================
REM Step 10: Check host at Argument5 and remote shutdown
REM ============================================================
:Step10_CheckAndShutdownRemote
  call :GetTimeStamp dtNow
  ping -n 2 %szRemoteIP% >nul 2>&1
  if !errorlevel! EQU 0 (
    echo [!dtNow!] Host %szRemoteIP% is reachable on the network.>> "%szLogFile%"
    shutdown /m \\%szRemoteIP% /s /t 0 >> "%szLogFile%" 2>&1
    if !errorlevel! EQU 0 (
      echo [!dtNow!] Shutdown command sent successfully.>> "%szLogFile%"
    ) else (
      echo [!dtNow!] Failed to send shutdown command ^(permissions/firewall/RPC^).>> "%szLogFile%"
    )
  ) else (
    echo [!dtNow!] Host %szRemoteIP% is NOT reachable.>> "%szLogFile%"
  )
  exit /b 0



REM ============================================================
REM Step 11: List computers on the local network
REM ============================================================
:Step11_ListNetworkHosts
  call :GetTimeStamp dtNow
  (
    echo [!dtNow!] List of network hosts ^(net view^):
    net view 2>&1
    echo [!dtNow!] ARP table ^(arp -a^):
    arp -a
  )>> "%szLogFile%"
  exit /b 0



REM ============================================================
REM Step 12: Check every IP address listed in ipon.txt
REM ============================================================
:Step12_CheckIpList
  call :GetTimeStamp dtNow
  if not exist "%szIpListFile%" (
    echo [!dtNow!] ipon.txt not found ^(%szIpListFile%^) - step skipped.>> "%szLogFile%"
    exit /b 0
  )
  set "boolAllUp=1"
  for /f "usebackq tokens=* delims=" %%A in ("%szIpListFile%") do (
    if not "%%A"=="" (
      ping -n 1 %%A >nul 2>&1
      if !errorlevel! EQU 0 (
        echo [!dtNow!]   %%A - reachable.>> "%szLogFile%"
```

```
      ) else (
        echo [!dtNow!]   %%A - NOT REACHABLE.>> "%szLogFile%"
        set "boolAllUp=0"
      )
    )
  )
  if "!boolAllUp!"=="0" (
    call :SendMail "Lab2: unreachable host" "One or more IPs from ipon.txt are unreachable. See %szLogFile% for
details."
  )
  exit /b 0



REM ============================================================
REM Step 13: Check current log file size
REM ============================================================
:Step13_CheckLogSize
  call :GetTimeStamp dtNow
  for /f %%S in ('powershell -NoProfile -Command "[math]::Round((Get-Item '%szLogFile%').Length/1KB)"') do set
"nLogSizeKB=%%S"
  echo [!dtNow!] Current log file size: !nLogSizeKB! KB ^(limit %nMaxLogKB% KB^).>> "%szLogFile%"
  if !nLogSizeKB! GTR %nMaxLogKB% (
    echo [!dtNow!] Log file size exceeded the limit!>> "%szLogFile%"
    call :SendMail "Lab2: log size exceeded" "Size !nLogSizeKB! KB exceeded the limit of %nMaxLogKB% KB."
  )
  exit /b 0



REM ============================================================
REM Step 14: Free / used space on all disks
REM ============================================================
:Step14_CheckDiskSpace
  call :GetTimeStamp dtNow
  (
    echo [!dtNow!] Disk space status:
    powershell -NoProfile -Command ^
      "Get-PSDrive -PSProvider FileSystem | Select-Object
Name,@{N='UsedGB';E={[math]::Round($_.Used/1GB,2)}},@{N='FreeGB';E={[math]::Round($_.Free/1GB,2)}} | Format-
Table -AutoSize"
  )>> "%szLogFile%" 2>&1
  exit /b 0



REM ============================================================
REM Step 15: Save systeminfo output to a file
REM ============================================================
:Step15_SaveSysteminfo
  call :GetTimeStamp dtNow
  for /f %%T in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd-HHmmss"') do set "szSysStamp=%%T"
  set "szSysFile=%szScriptDir%systeminfo-%szSysStamp%.txt"
  systeminfo > "%szSysFile%" 2>&1
  echo [!dtNow!] systeminfo output saved to %szSysFile%.>> "%szLogFile%"
  exit /b 0



REM ============================================================
REM Step 16: Task Scheduler hint for recurring auto-start
REM ============================================================
:Step16_ShowSchedulerHint
  call :GetTimeStamp dtNow
  (
    echo [!dtNow!] For recurring auto-start, use Task Scheduler, e.g.:
    echo [!dtNow!]   schtasks /create /tn "LR2_Script" /tr "\"%szScriptDir%lab.cmd\" lab.log d:\baza xx.exe
d:\archive 192.168.1.2 500" /sc daily /st 09:00 /rl highest
```

```
  )>> "%szLogFile%"
  exit /b 0



REM ============================================================
REM Helper: send an email via Send-MailMessage
REM Params: %1 - subject, %2 - body
REM ============================================================
:SendMail
  if "%szSmtpServer%"=="" (
    call :GetTimeStamp dtNow
    echo [!dtNow!] SMTP is not configured - email "%~1" NOT sent ^(see settings block at the top of the
script^).>> "%szLogFile%"
    exit /b 0
  )
  powershell -NoProfile -Command ^
    "$pass = ConvertTo-SecureString '%szSmtpPass%' -AsPlainText -Force; ^
     $cred = New-Object System.Management.Automation.PSCredential('%szSmtpUser%', $pass); ^
     Send-MailMessage -SmtpServer '%szSmtpServer%' -Port %nSmtpPort% -UseSsl -Credential $cred -From
'%szSmtpFrom%' -To '%szSmtpTo%' -Subject '%~1' -Body '%~2'" ^
    >> "%szLogFile%" 2>&1
  exit /b 0



REM ============================================================
REM Helper: returns the current date/time as a string
REM Param: %1 - variable name to store the result in
REM ============================================================
:GetTimeStamp
  for /f %%T in ('powershell -NoProfile -Command "Get-Date -Format \"yyyy-MM-dd HH:mm:ss\""') do set "%~1=%%T"
  exit /b 0

REM ============================================================
REM End of file lab.cmd
REM ============================================================
```

8. Приклад фрагменту log-файлу

Наведений нижче фрагмент ілюструє очікуваний вигляд журналу після одного запуску скрипта (реальний вміст залежить від стану конкретної машини на момент запуску; для звіту потрібно вставити фактичний фрагмент lab.log зі свого запуску):

```
[2026-09-20 10:00:01] Log file "lab.log" opened or created.
[2026-09-20 10:00:03] Time synchronization: success.
[2026-09-20 10:00:03]   Old time: 2026-09-20 09:59:58
[2026-09-20 10:00:03]   New time: 2026-09-20 10:00:03
[2026-09-20 10:00:05] Process xx.exe was not found among running processes.
[2026-09-20 10:00:06] Temporary files deleted: 4 (from d:\baza).
[2026-09-20 10:00:12] Archive 20260920-100010.zip created and copied to d:\archive.
[2026-09-20 10:00:12] Archive for the previous day (20260919) was found.
[2026-09-20 10:00:13] Old archives found and deleted (older than 30 days): 1.
[2026-09-20 10:00:15] Internet connection: available (ping 8.8.8.8 succeeded).
[2026-09-20 10:00:16] Host 192.168.1.2 is NOT reachable.
```

9. Скріншоти виконання роботи

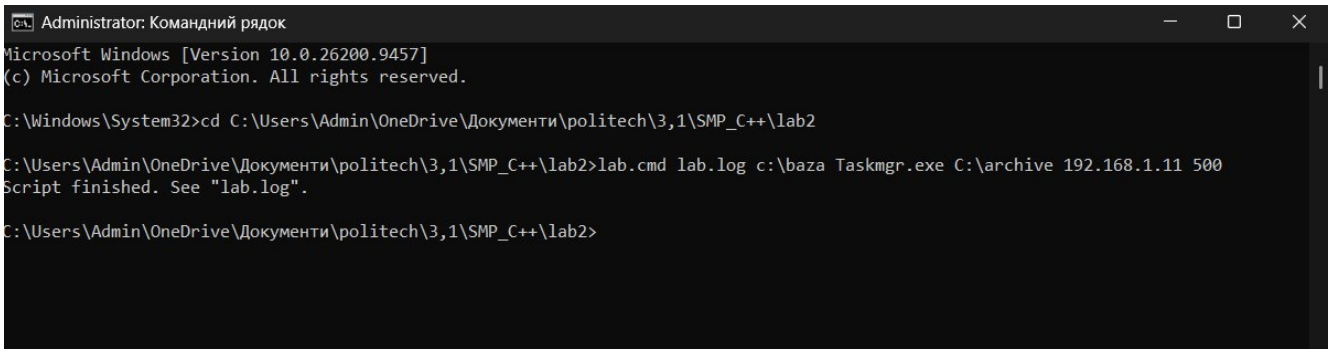


Рис.1 Результат виконання в консолі

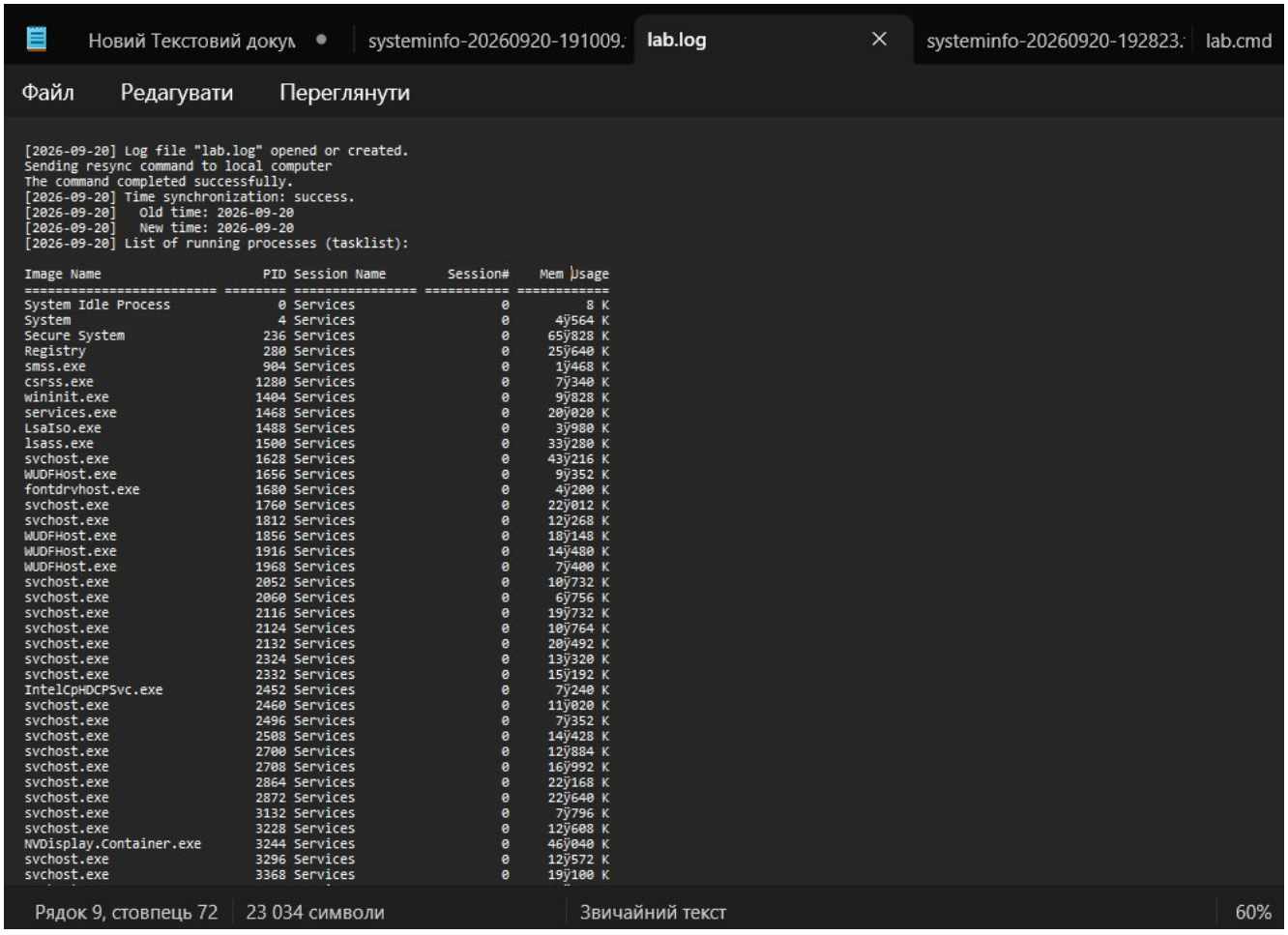


Рис.2 Результат виконання в lab.log

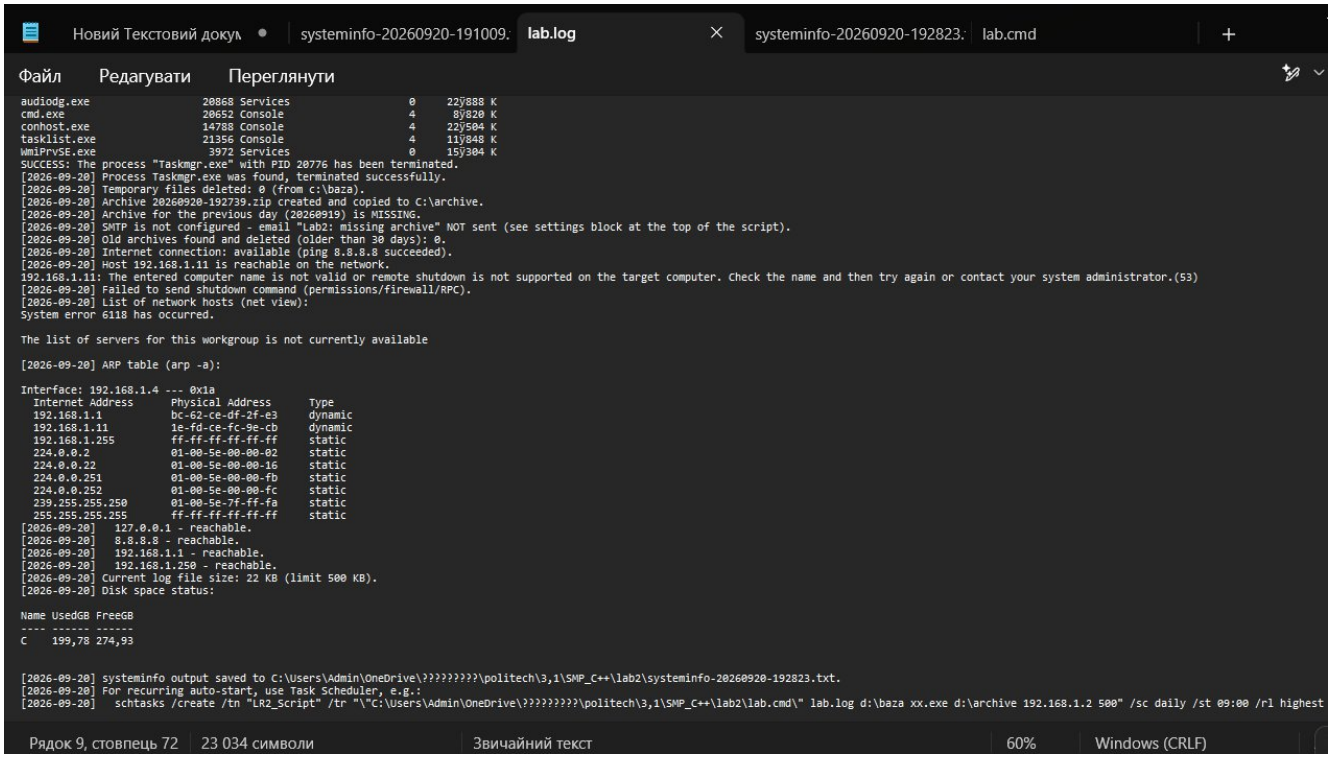


Рис.3 Результат виконання в lab.log

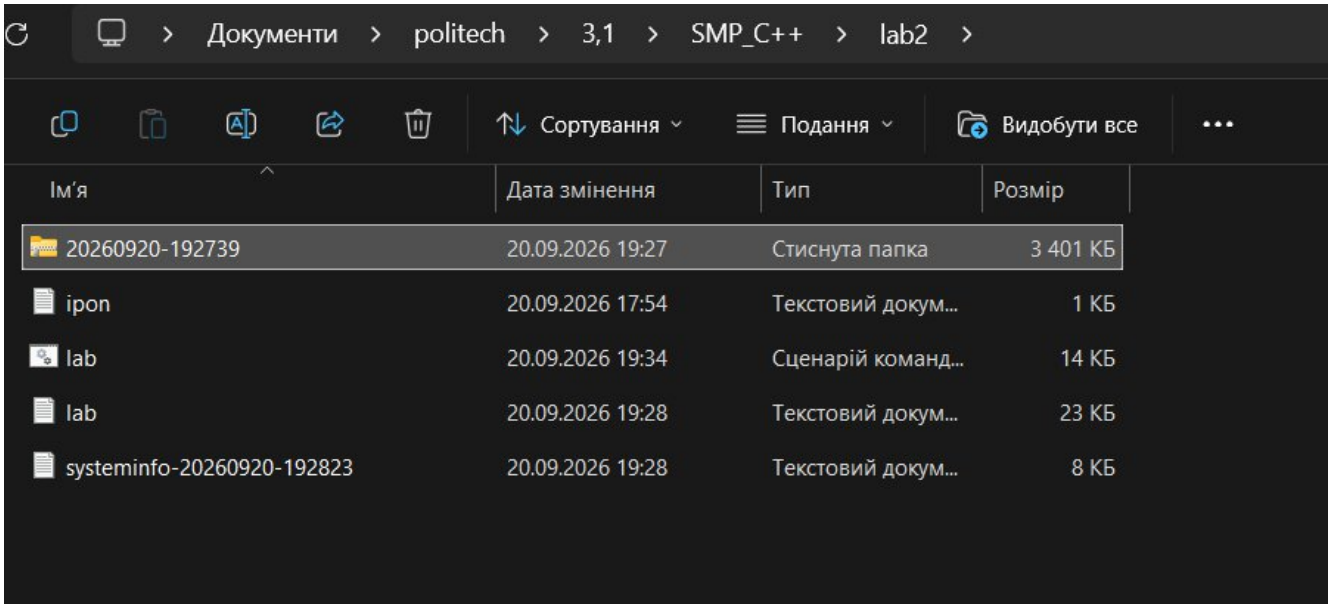


Рис.4 Результат виконання в головній папці проекту (створено архів lab.log та systeminfo...)

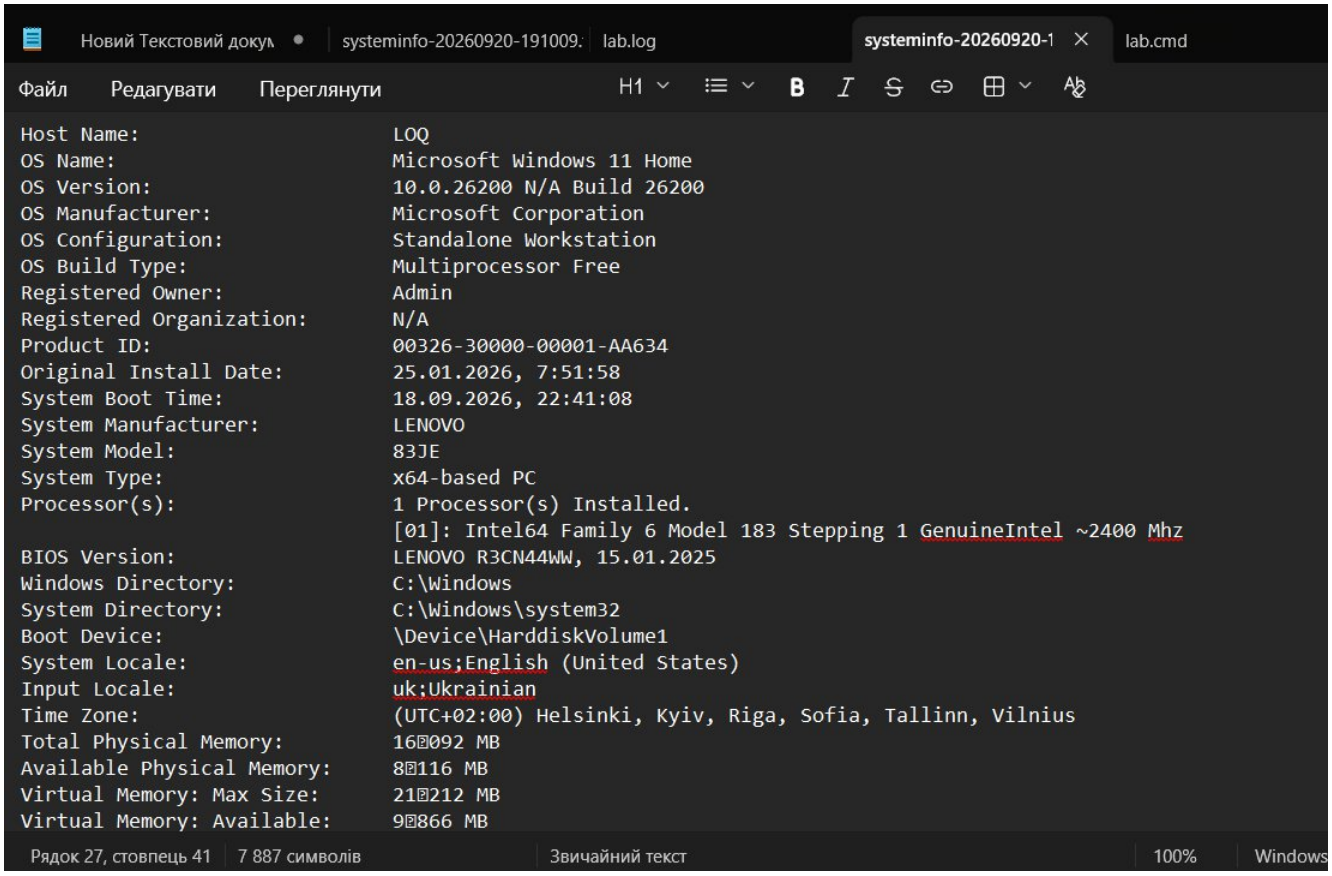


Рис.5 Результат виконання в systeminfo

10. Планувальник завдань (автозапуск)

Приклад команди створення регулярного щоденного завдання о 9:00 з правами найвищого рівня:

```
schtasks /create /tn "LR2_Script" /tr "\"C:\path\lab.cmd\" lab.log d:\baza xx.exe d:\archive 192.168.1.2 500" /sc daily /st 09:00 /rl highest
```

Альтернативно завдання можна створити вручну через графічний інтерфейс Планувальника завдань (taskschd.msc), вказавши тригер (Daily) та дію (Start a program → lab.cmd з відповідними аргументами).

Висновки

У результаті виконання лабораторної роботи розроблено пакетний файл lab.cmd, що автоматизує типовий набір завдань системного адміністрування Windows: ведення журналу, синхронізацію часу, аналіз і завершення процесів, очищення тимчасових файлів, архівування з ротацією застарілих архівів, мережну діагностику, віддалене вимкнення ПК, контроль дискового простору та розміру журналу, а також збереження системної інформації. Під час роботи закріплено практичні навички роботи із внутрішніми та зовнішніми командами CMD, циклами й умовними операторами batch-мови, а також інтеграцією PowerShell-команд у пакетні скрипти.